项目讲解 / 微博式合成级联

从转发树看社交网络行为

一条原帖发出后，有人直接转发，有人从别人的转发继续转发。把这些关系画成传播树（propagation tree），就能比较传播形态、区分事件类别，并预测下一条转发可能来自哪里。

20,648
合成传播图
训练、验证、测试合计

0.794
分类宏平均 F1
图结构＋节点向量

49.1%
来源预测 Hits@5
真实来源排进前五

同一份数据，三个问题

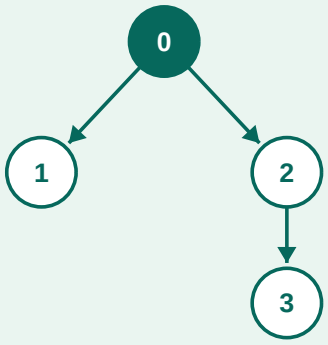
01	看形态	传播是集中在原帖，还是往下接力了好几层?
02	判类别	给模型整棵树，让它输出这张图属于哪一类。
03	找来源	只给模型已经出现的部分，让它给候选来源排序。

数据来源

帖子、关系、数值向量和类别均由模拟器（simulator）生成。报告里的分数描述的是同一生成机制下的新样本。

把转发记录读成一棵树

一个节点（node）是一条模拟帖子，一条有向边（directed edge）从来源指向转发帖。



例图 0 是原帖。1 和 2 来自 0，3 来自 2。

`edge_index` 两行分别记录 [0,0,2] 和 [1,2,3]

`x` 每条帖子 32 个模拟数值，例图为 4×32

`label` 整张树的类别，0 正常，1 突发

三份数据各有多少样本

划分（split）	总图数	正常	突发
训练集	14,453	9,633	4,820
验证集	3,097	2,064	1,033
测试集	3,098	2,065	1,033

训练集里的平均传播形态

图特征（graph feature）	正常	突发
节点数（node count）	42.44	42.70
最深转发层数（maximum depth）	4.90	4.10
最宽一层占比（maximum width fraction）	39.9%	51.1%
直接来自原帖的比例（root child fraction）	38.2%	51.6%
没有下游的节点占比（leaf fraction）	63.0%	69.5%

两类图平均都是约 42 个节点，突发类有更多转发直接接在原帖下。每棵树的边数固定为 N−1，有向图密度（directed density）等于 1/N，只反映图的大小。

给整棵树判一个类别

把完整的传播树交给分类器（classifier），输出它属于突发类的分数。

逻辑回归（logistic regression）先把每张图变成一行数。结构部分是七项图特征，向量部分是图内所有节点的 32 维均值。两部分可以分别使用，也可以拼成 39 维。

图同构网络（GIN）相邻节点先双向交换信息，两轮后对节点表示取均值，再对整张图打分。验证集用来选训练轮次，测试集用来计算下表。

测试集 / 3,098 张图

方法	准确率	宏平均 F1	突发召回率	ROC-AUC
多数类（全部判正常）	0.667	0.400	0.000	0.500
仅节点数	0.667	0.400	0.000	0.488
图结构	0.731	0.664	0.428	0.766
节点向量均值	0.777	0.733	0.556	0.835
图结构＋节点向量均值	0.822	0.794	0.674	0.886
图同构网络（GIN）	0.796	0.760	0.612	0.850

准确率（accuracy）统计所有预测的正确比例。宏平均 F1（macro-F1）先分别算正常类和突发类的 F1，再给两类同等权重。F1 同时考虑查准和查全。两者一起看，就能发现多数类策略漏掉了全部突发事件。

结构＋向量模型怎么错

真实类别	预测正常	预测突发
正常	1,852	213
突发	337	696

突发召回率（burst recall）为 696 ÷ 1,033 = 67.4%。合用两种输入在本次测试中得分最高。合成向量本身带有生成器设定的类别差异。

一条新转发，可能来自哪?

链接的两端是同一传播树内的帖子。模型给已出现的帖子排序。

1	先看一段	把最早的 70% 节点和它们的边作为已知前缀（observed prefix）。
2	给候选打分	例如节点 8 刚到，候选是节点 0 到 7。计算它们已有的出度和深度。
3	再揭示答案	记下真实来源在榜单上的名次，加入这条边，再预测下一条。

逻辑回归（logistic regression）根据根节点标记、已有出度（out-degree）、深度（depth）和顺序间隔打分。训练时每次取真实来源和最多 8 个随机负样本（negative samples），测试时为全部候选排序。

测试集 / 37,936 次排序

方法	Hits@1	Hits@5	MRR
均匀随机	2.8%	14.1%	0.115
越近越优先	2.6%	12.2%	0.101
已有出度优先	34.7%	47.0%	0.423
逻辑回归（logistic regression）	35.2%	49.1%	0.434

Hits@5 表示真实来源排进前五的比例。平均倒数排名（MRR）把第 1 名记作 1，第 2 名记作 1/2，再对所有预测取平均。并列名次按等概率打破来计分。

首位命中来自哪里

真实来源是非根节点（non-root node）的 24,600 次预测中，逻辑回归的首位命中率为 0.0%，前五命中率为 21.5%。这解释了为什么总体首位命中率需要结合根节点优势来看。

自己把实验跑一遍

从仓库根目录开始。使用 Python 3.10 或更高版本。

```
cd experiments/weibo_cascades

python -m pip install -r requirements.txt

python generate_dataset.py --out data

python run_experiments.py
```

生成脚本（generator）已核验能复现上传文件中的节点向量、父节点和标签顺序。若已有三份 .pt 文件，可以跳过生成，给最后一条命令加上 --data-dir 数据目录。

运行后看这些文件

`metrics.json`	两项任务的指标、数据内容指纹和 GIN 训练记录
`event_test_predictions.csv`	每张测试图的真实类别和各模型的突发分数
`link_test_predictions.csv`	每条待预测边的真实来源、首选来源和排名
`gin_model.pt`	验证集宏平均 F1 最佳轮次的模型权重

换成真实数据，要补什么

用户互动预测（user interaction prediction） 需要能跨事件匹配的用户编号，以及带时间戳（timestamp）的转发、评论或提及记录。节点编号在本实验里只表示一张树内的生成次序。

突发事件预警（early event detection） 需要明确观测截止点（observation cutoff），只输入该时点以前的数据，并用独立的突发定义标注结果。这里的事件分类使用完整传播树。

仓库中的 experiments/weibo_cascades/README.md 有参数说明，results/weibo_cascades/metrics.json 保留了本次实验的完整数值。